Aprendizaje por Refuerzo

Métodos de n-Pasos

Luis Daniel Hernández Molinero
Ciencia de la Computación e Inteligencia Artificial
Dept. Ingeniería de la Información y las Comunicaciones

Universidad de Murcia

Máster en Inteligencia Artificial

2025 / 2026

Índice de Contenidos

1. Introducción

2. Predicción TD de n pasos

3. Control TD de n pasos
   3.1. Aprendizaje On-Policy de n Pasos
   3.2. Aprendizaje Off-Policy de n Pasos
      Basado en Muestreo por Importancia
      Sin Muestreo por Importancia

4. Referencias

Desarrollo

1. Introducción

2. Predicción TD de n pasos

3. Control TD de n pasos
   3.1. Aprendizaje On-Policy de n Pasos
   3.2. Aprendizaje Off-Policy de n Pasos
      Basado en Muestreo por Importancia
      Sin Muestreo por Importancia

4. Referencias

Introducción

- Un **resumen** de los métodos MC y TD(0) Hemos estudiado:

Característica	Monte Carlo (MC)	TD(0)
Requiere episodios completos	Sí, requiere un episodio completo	No, se actualiza paso a paso
Varianza	Alta, debido a las fluctuaciones del retorno	Baja, ya que se basa en predicciones parciales
Eficiencia	Puede ser menos eficiente en episodios largos	Más eficiente, actualiza continuamente
Estabilidad	Más estable en entornos con retorno claro	Menos estable debido a la propagación de errores
Aplicabilidad	No es adecuado para entornos no episódicos	Adecuado para entornos no episódicos

- **Los métodos n-step TD generalizan** los métodos **MC** y **TD** para que uno pueda pasar suavemente de uno a otro según sea necesario para cumplir con las demandas de una tarea en particular.
- Los métodos **MC** están en un extremo del espectro y los métodos **TD de un solo paso** en el otro.
- **Los mejores métodos** suelen ser aquellos que se encuentran entre estos dos extremos.
  - Conviene explorar en intervalo de tiempo más amplios.
- **Estudiaremos** primero la predicción y después el control

Desarrollo

1. Introducción

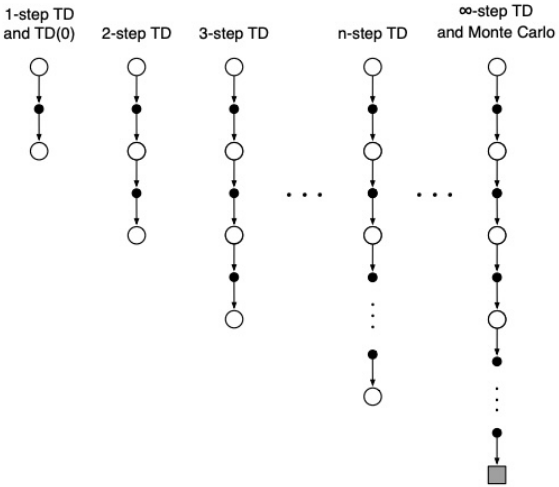
2. Predicción TD de n pasos

3. Control TD de n pasos
   3.1. Aprendizaje On-Policy de n Pasos
   3.2. Aprendizaje Off-Policy de n Pasos
      Basado en Muestreo por Importancia
      Sin Muestreo por Importancia

4. Referencias

Predicción n-step TD

- Realizar una actualización basada en **una secuencia de n recompensas**:
  - Más de una, pero menos que todas ellas.
- **Gráficamente**, se pueden representan las siguientes variantes de TD:



Predicción n-step TD (Actualización)

- Recuerda: $NuevoEstimado = ViejoEstimado + TamañodelPaso \cdot (Objetivo - ViejoEstimado)$.
- El **objetivo** (target) para una actualización arbitraria de n-step es el **retorno n-step**:

$$
\begin{aligned}
G_{t:t+1} &\doteq R_{t+1} + \gamma V_t(S_{t+1}) \\
G_{t:t+2} &\doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{t+1}(S_{t+2}), \\
&\cdots \\
G_{t:t+n} &\doteq R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n}) \quad \forall n \geq 1, \forall t \text{ t.q } 0 \leq t < T - n \\
G_{t:t+n} &\doteq G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T \text{ si } t + n \geq T
\end{aligned}
$$

- **Los retornos n-step** pueden considerarse **aproximaciones** del retorno completo, **truncadas** después de n pasos y luego corregidas por los términos faltantes.
- La fórmula para la **actualización n-step** es: $V_{t+n}(S_t) = V_{t+n-1}(S_t) + \alpha\,(G_{t:t+n} - V_{t+n-1}(S_t))$
- Mientras que los valores de todos los demás estados permanecen inalterados:

$$V_{t+n}(s) = V_{t+n-1}(s) \quad \text{para todos } s \neq S_t$$

- Una **expresión equivalente** es la siguiente:

$$V(S_t) \leftarrow V(S_t) + \alpha \left[\sum_{k=1}^{n} \gamma^{k-1} R_{t+k} + \gamma^n V(S_{t+n}) - V(S_t)\right]$$

DT de n-pasos para estimar v_π
Sutton y Barto, 2018

$$V(S_t) \leftarrow V(S_t) + \alpha \left[\sum_{k=1}^{n} \gamma^{k-1} R_{t+k} + \gamma^n V(S_{t+n}) - V(S_t) \right]$$

Algorithm 1 DT de n-pasos para estimar $V \approx v_\pi$

n-step TD for estimating $V \approx v_\pi$

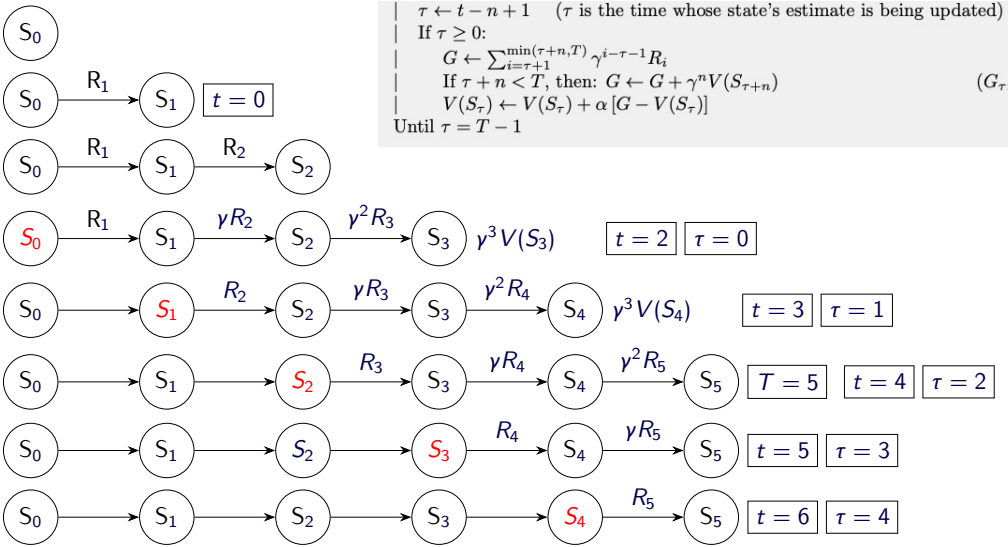
Input: a policy π
Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n
Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$
All store and access operations (for S_t and R_t) can take their index mod $n + 1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
  | If $t < T$, then:
  | Take an action according to $\pi(\cdot|S_t)$
  | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
  | If S_{t+1} is terminal, then $T \leftarrow t + 1$
  | $\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)
  | If $\tau \geq 0$:
  | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
  | If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$ $(G_{\tau:\tau+n})$
  | $V(S_\tau) \leftarrow V(S_\tau) + \alpha [G - V(S_\tau)]$
  Until $\tau = T - 1$

Un ejemplo para 3 pasos

$$G_{t:t+3} \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 V_{t+2}(S_{t+3})$$

| $\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)
| If $\tau \geq 0$:
| $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
| If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$ $(G_{\tau:\tau+n})$
| $V(S_\tau) \leftarrow V(S_\tau) + \alpha\,[G - V(S_\tau)]$
Until $\tau = T - 1$



Propiedades de los Retornos de n Pasos en TD

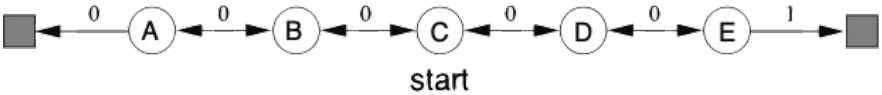
- **Retornos de n pasos** utilizan la función de valor V_{t+n-1} para corregir las recompensas faltantes más allá de R_{t+n}.
- El retorno de n pasos toma en cuenta más información que solo el valor $V_{t+n-1}(s)$, ya que incluye las recompensas obtenidas durante los n pasos y una estimación de los valores futuros (a través de $V_{t+n-1}(s_{t+n})$).
- La propiedad de reducción de error garantiza que el peor error del retorno esperado de n pasos es:

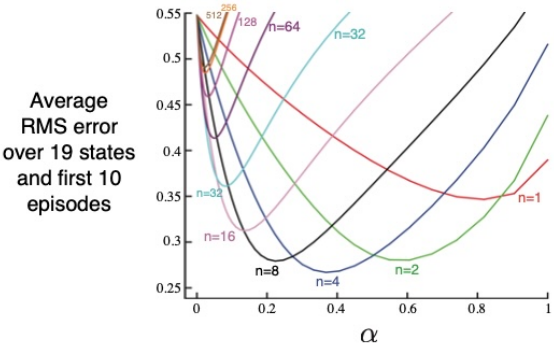
$$\max_s \left| E_\pi[G_{t:t+n} | S_t = s] - v_\pi(s) \right| \leq \gamma^n \max_s \left| V_{t+n-1}(s) - v_\pi(s) \right|$$

- **Convergencia**: Los métodos TD de n pasos convergen a las predicciones correctas bajo condiciones técnicas adecuadas.
- Los métodos TD de n pasos son una **familia sólida de métodos**, con los métodos TD de un solo paso y Monte Carlo como miembros extremos.

Pero depende de α

Ejemplo: ¿Qué valor de n es mejor?
n pasos en el Random Walk





- 19 estados, saliendo por la izquierda con -1.
- Rendimiento de los métodos TD de n pasos en función de γ, para varios valores de n, en una tarea de random walk de 19 estados.
- La medida de rendimiento es la raíz cuadrada del error cuadrada promedio entre las predicciones al final del episodio para los 19 estados y sus valores verdaderos, entonces se promedian sobre los primeros 10 episodios y se realizan 100 repeticiones de todo el experimento (se usaron los mismos conjuntos de caminatas para todas las configuraciones de parámetros).

Desarrollo

1. Introducción

2. Predicción TD de n pasos

3. Control TD de n pasos
   3.1. Aprendizaje On-Policy de n Pasos
   3.2. Aprendizaje Off-Policy de n Pasos
      Basado en Muestreo por Importancia
      Sin Muestreo por Importancia

4. Referencias

Profundizamos en ...

1. Introducción

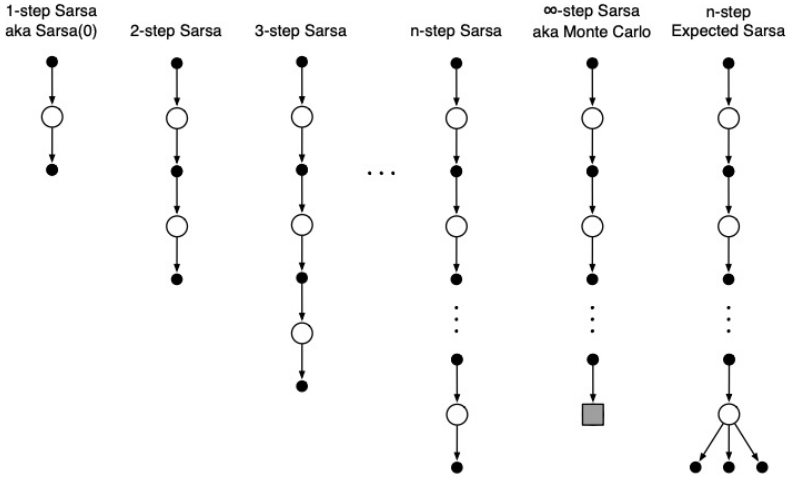
2. Predicción TD de n pasos

3. Control TD de n pasos
   3.1. Aprendizaje On-Policy de *n* Pasos
   3.2. Aprendizaje Off-Policy de *n* Pasos
      Basado en Muestreo por Importancia
      Sin Muestreo por Importancia

4. Referencias

Variantes de SARSA

- Realizar una actualización basada en una **secuencia de n recompensas**:
  - Más de una, pero menos que todas ellas.
- **Gráficamente**, se representan las siguientes variantes de Sarsa:



Sarsa's de n-pasos

- La **versión original** es Sarsa de un solo paso, o Sarsa(0).

$$Q(S_t, A_t) \leftarrow Q(\underline{S_t, A_t}) + \alpha \Big(\underline{R_{t+1}} + \gamma Q(\underline{S_{t+1}, A_{t+1}}) - Q(S_t, A_t) \Big)$$

- La *versión de n pasos* de Sarsa la llamamos **n-step Sarsa**.
  - El **objetivo** (target) para una actualización arbitraria de n-step es el **retorno n-step**:

$$\begin{aligned}
G_{t:t+1} &\doteq R_{t+1} + \gamma Q_t(S_{t+1}, A_{t+1}) \\
G_{t:t+2} &\doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 Q_{t+1}(S_{t+2}, A_{t+2}) \\
&\cdots \\
G_{t:t+n} &\doteq R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n Q_{t+n-1}(S_{t+n}, A_{t+n}), \text{ para } n \geq 1 \text{ y } 0 \leq t < T \\
G_{t:t+n} &\doteq G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T \text{ si } t + n \geq T
\end{aligned}$$

  - La **fórmula de actualización** es: $Q_{t+n}(S_t, A_t) = Q_{t+n-1}(S_t, A_t) + \alpha \big(G_{t:t+n} - Q_{t+n-1}(S_t, A_t) \big)$
- **n-step Expected SARSA** es la versión de n pasos de Expected Sarsa
  - Se redefine el **objetivo** (target), llamado **retorno n-step**, y se usa fórmula de actualización anterior:
    $G_{t:t+n} \doteq R_{t+1} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n \overline{V}_{t+n-1}(S_{t+n}),\ t + n < T$ y $G_{t:t+n} \doteq G_t$ para $t + n \geq T$
    donde $\overline{V}_t(s) \doteq \sum_a \pi(a|s) Q_t(s, a), \quad \forall s \in S$.

SARSA de n-pasos para estimar $Q \approx q_\pi$

$$Q_{t+n}(S_t, A_t) = Q_{t+n-1}(S_t, A_t) + \alpha\left(G_{t:t+n} - Q_{t+n-1}(S_t, A_t)\right)$$

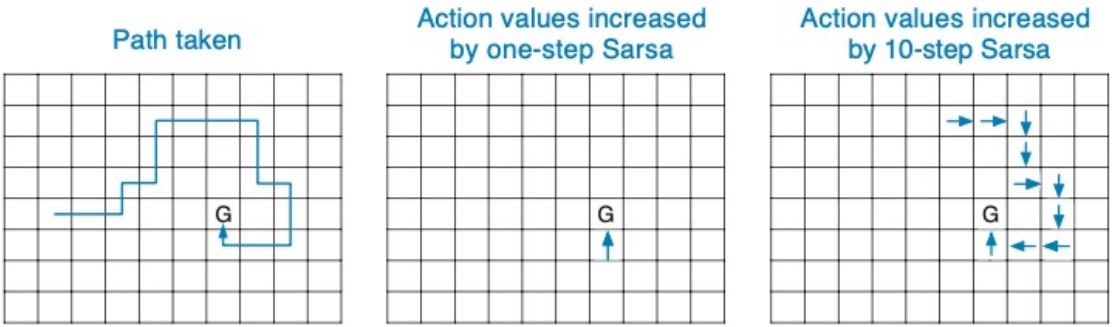
n-step Sarsa for estimating $Q \approx q_*$ or q_π

Initialize $Q(s,a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize π to be ε-greedy with respect to Q, or to a fixed given policy
Algorithm parameters: step size $\alpha \in (0,1]$, small $\varepsilon > 0$, a positive integer n
All store and access operations (for S_t, A_t, and R_t) can take their index mod $n+1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  Select and store an action $A_0 \sim \pi(\cdot|S_0)$
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
  | If $t < T$, then:
  | Take action A_t
  | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
  | If S_{t+1} is terminal, then:
  | $T \leftarrow t+1$
  | else:
  | Select and store an action $A_{t+1} \sim \pi(\cdot|S_{t+1})$
  | $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
  | If $\tau \geq 0$:
  | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
  | If $\tau + n < T$, then $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ $(G_{\tau:\tau+n})$
  | $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha\left[G - Q(S_\tau, A_\tau)\right]$
  | If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is ε-greedy wrt Q
  Until $\tau = T - 1$

Convergencia de Q:
$\max_s \max_a |Q_{\text{new}}(s,a) - Q_{\text{old}}(s,a)| < \delta$
$\max_s |\pi_{\text{new}}(s) - \pi_{\text{old}}(s)| < \delta$

Ejemplo de SARSA de n-pasos



- **Izquierda:** Un episodio.
- **Centro:** SARSA a 1 paso. Solo la última acción de la secuencia de acciones actualiza la recompensa.
- **Derecha:** SARSA a 10 pasos. Las últimas 10 acciones de la secuencia de acciones actualizan sus valores. Aprende mucho más rápido en un solo episodio.

Profundizamos en ...

1. Introducción

2. Predicción TD de n pasos

3. Control TD de n pasos
   3.1. Aprendizaje On-Policy de n Pasos
   3.2. Aprendizaje Off-Policy de n Pasos
      Basado en Muestreo por Importancia
      Sin Muestreo por Importancia

4. Referencias

Con más detalle ..

1. Introducción

2. Predicción TD de n pasos

3. Control TD de n pasos
   3.1. Aprendizaje On-Policy de n Pasos
   3.2. Aprendizaje Off-Policy de n Pasos
      Basado en Muestreo por Importancia
      Sin Muestreo por Importancia

4. Referencias

Recordatorio MC

- En **Monte Carlo** estudiamos el muestreo por importancia ordinario y ponderado.

$$V(s) \doteq \frac{\sum_{t\in\mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{|\mathcal{T}(s)|} \qquad V(s) \doteq \frac{\sum_{t\in\mathcal{T}(s)} \rho_{t:T(t)-1} G_t}{\sum_{t\in\mathcal{T}(s)} \rho_{t:T(t)-1}}, \qquad \text{con } \rho_{t:T(t)-1} \doteq \prod_{k=t}^{T(t)-1} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}$$

- En su momento vimos que las funciones $Q(s,a)$ tienen un **cálculo incremental**:

$$Q(S_t,A_t) \leftarrow Q(S_t,A_t) + \frac{\rho_{t:T(t)-1}}{C(S_t,A_t)}[G_t - Q(S_t,A_t)] = Q(S_t,A_t) + \alpha\rho_{t:T(t)-1}[G_t - Q(S_t,A_t)]$$

- **En métodos de n pasos**, estamos interesados en
  - La ganancia para n-pasos:
    - $G_{t:t+n} = \underbrace{R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n}}_{(n-1)-recompensas} + \underbrace{\gamma^n FuncionValor}_{Recompensa-Truncada}$, para $n \geq 1$ y $0 \leq t+n < T$
    - $G_{t:t+n} = G_{t:T}$ si $t+n \geq T$
  - la probabilidad relativa de solo n acciones. $\rho_{t:\text{lím}} = \prod_{k=t}^{\text{lím}} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}$

Aprendizaje Off-Policy de n Pasos

- Definiremos la **razón de muestreo de importancia** de tomar las n-acciones (desde t hasta h):

$$\rho_{t:h} \doteq \prod_{k=t}^{\min(h,T-1)} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}$$

  - Donde T hace referencia al instante de llegar al estado final.
- **Para valores de estado**, definiremos la función de actualización como

$$V_{t+n}(S_t) \doteq V_{t+n-1}(S_t) + \alpha\ \rho_{t:t+n-1}\ \left[G_{t:t+n} - V_{t+n-1}(S_t)\right] \quad 0 \leq t < T$$

  - $G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n})$
  - La razón de muestreo de importancia de tomar las n acciones va desde A_t hasta A_{t+n-1}
- **Para valores de acción**, definiremos la función de actualización como

$$Q_{t+n}(S_t, A_t) \doteq Q_{t+n-1}(S_t, A_t) + \alpha\ \rho_{t+1:t+n}\ \left[G_{t:t+n} - Q_{t+n-1}(S_t, A_t)\right], \quad 0 \leq t < T,$$

  - $G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n Q_{t+n-1}(S_{t+n}, A_{t+n})$
  - La razón de muestreo de importancia de tomar las n acciones va desde A_t hasta A_{t+n-1}
- **Si las dos políticas son las mismas**, el ratio de muestreo por importancia es 1.
  - $V_{t+n}(S_t)$ generaliza la actualización TD de n-pasos.
  - $Q_{t+n}(S_t, A_t)$ generaliza la actualización Sarsa de n-pasos.
- Para la versión off-policy de **n-step Expected Sarsa**:
  - $G_{t:t+n} \doteq R_{t+1} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n \overline{V}_{t+n-1}(S_{t+n})$ donde $\overline{V}_t(s) \doteq \sum_a \pi(a|s) Q_t(s,a)$.
  - Usará la función Q con $\rho_{t+1:t+n-1}$ en lugar de $\rho_{t+1:t+n}$

SARSA de n-pasos off-policy para estimar $Q \approx q_\pi$

$$Q_{t+n}(S_t, A_t) \doteq Q_{t+n-1}(S_t, A_t) + \alpha \rho_{t+1:t+n}\big[G_{t:t+n} - Q_{t+n-1}(S_t, A_t)\big]$$

Off-policy n-step Sarsa for estimating $Q \approx q_*$ or q_π

Input: an arbitrary behavior policy b such that $b(a|s) > 0$, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize $Q(s,a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize π to be greedy with respect to Q, or as a fixed given policy
Algorithm parameters: step size $\alpha \in (0,1]$, a positive integer n
All store and access operations (for S_t, A_t, and R_t) can take their index mod $n+1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  Select and store an action $A_0 \sim b(\cdot|S_0)$
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
  | If $t < T$, then:
  | Take action A_t
  | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
  | If S_{t+1} is terminal, then:
  | $T \leftarrow t+1$
  | else:
  | Select and store an action $A_{t+1} \sim b(\cdot|S_{t+1})$
  | $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
  | If $\tau \geq 0$:
  | $\rho \leftarrow \prod_{i=\tau+1}^{\min(\tau+n,T-1)} \frac{\pi(A_i|S_i)}{b(A_i|S_i)}$ $(\rho_{\tau+1:\tau+n})$
  | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
  | If $\tau + n < T$, then: $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ $(G_{\tau:\tau+n})$
  | $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha\rho\left[G - Q(S_\tau, A_\tau)\right]$
  | If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is greedy wrt Q
  Until $\tau = T - 1$

Con más detalle ..

1. Introducción

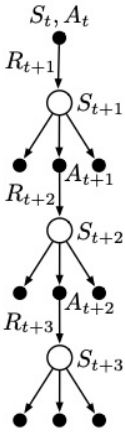
2. Predicción TD de n pasos

3. Control TD de n pasos
   3.1. Aprendizaje On-Policy de n Pasos
   3.2. Aprendizaje Off-Policy de n Pasos
      Basado en Muestreo por Importancia
      Sin Muestreo por Importancia

4. Referencias

Idea Intuitiva de *n-step Tree Backup Algorithm*

- ¿Es posible el aprendizaje off-policy sin muestreo de importancia?
- Árbol de respaldo para 3 acciones:
  - Hay 3 muestras de estados y recompensas, y 2 muestras de acciones
  - Las v.a. representan los **eventos que ocurren después** del par inicial estado-acción (S_t, A_t).
  - Colgadas a los lados de cada estado están las **acciones que no fueron seleccionadas**.
  - Para el último estado, todas las acciones se consideran como *acciones que no han sido (todavía) seleccionadas*.
    - Debido a que no tenemos datos de muestra para las acciones no seleccionadas, realizamos *bootstrap* y utilizamos las estimaciones de sus valores para formar el objetivo para la actualización.
- Cada nodo hoja contribuye al objetivo con un peso proporcional a su probabilidad de ocurrir bajo la política objetivo π.
  - Cada acción de primer nivel a contribuye con un peso de $\pi(a|S_{t+1})$.
  - La acción realmente tomada, A_t, no contribuye en absoluto. Su probabilidad, $\pi(A_{t+1}|S_{t+1})$, se utiliza para ponderar todos los valores de acción del segundo nivel.
  - Cada acción de segundo nivel no seleccionada, a', contribuye con peso $\pi(A_{t+1}|S_{t+1})\pi(a'|S_{t+2})$.
  - Cada acción de tercer nivel contribuye con peso $\pi(A_{t+1}|S_{t+1})\pi(A_{t+2}|S_{t+2})\pi(a''|S_{t+3})$, y así sucesivamente.



the 3-step tree-backup update

Justificación Matemática

- El retorno de un solo paso (objetivo) lo vamos a considera igual que Expected Sarsa:

$$G_{t:t+1} = R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q_t(S_{t+1}, a) \text{ si } t < T-1 \; ; \qquad G_{T-1:T} = R_T \quad \text{si} \quad \text{t = T-1}$$

- El retorno de dos pasos es para $t < T-2$

$$G_{t:t+2} \doteq R_{t+1} + \gamma \sum_{a \neq A_{t+1}} \pi(a \mid S_{t+1}) Q_{t+1}(S_{t+1}, a) + \gamma \pi(A_{t+1} \mid S_{t+1}) \left(R_{t+2} + \gamma \sum_a \pi(a \mid S_{t+2}) Q_{t+1}(S_{t+2}, a) \right)$$

$$= R_{t+1} + \gamma \sum_{a \neq A_{t+1}} \pi(a \mid S_{t+1}) Q_{t+1}(S_{t+1}, a) + \gamma \pi(A_{t+1} \mid S_{t+1}) G_{t+1:t+2},$$

- El retorno para n-pasos, $t < T-1, n \geq 2$:

$$G_{t:t+n} \doteq R_{t+1} + \gamma \sum_{a \neq A_{t+1}} \pi(a|S_{t+1}) Q_{t+n-1}(S_{t+1}, a) + \gamma \pi(A_{t+1}|S_{t+1}) G_{t+1:t+\mathbf{n}}$$

- Regla de actualización de valor-acción de n-pasos Sarsa:

$$Q_{t+n}(S_t, A_t) \doteq Q_{t+n-1}(S_t, A_t) + \alpha \left[G_{t:t+n} - Q_{t+n-1}(S_t, A_t) \right],$$

para $0 \leq t < T$, mientras que los valores de todos los demás pares estado-acción permanecen sin cambios: $Q_{t+n}(s, a) = Q_{t+n-1}(s, a)$, para todos s, a tales que $s \neq S_t$ o $a \neq A_t$.

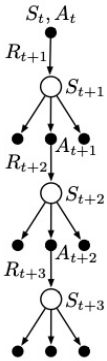
Árbol de Respaldo de n-pasos para estimar $Q \approx q_\pi$

$$Q_{t+n}(S_t, A_t) \doteq Q_{t+n-1}(S_t, A_t) + \alpha\big[G_{t:t+n} - Q_{t+n-1}(S_t, A_t)\big]$$

n-step Tree Backup for estimating $Q \approx q_*$ or q_π

Initialize $Q(s,a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize π to be greedy with respect to Q, or as a fixed given policy
Algorithm parameters: step size $\alpha \in (0,1]$, a positive integer n
All store and access operations can take their index mod $n+1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  Choose an action A_0 arbitrarily as a function of S_0; Store A_0
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
  | If $t < T$:
  | Take action A_t; observe and store the next reward and state as R_{t+1}, S_{t+1}
  | If S_{t+1} is terminal:
  | $T \leftarrow t+1$
  | else:
  | Choose an action A_{t+1} arbitrarily as a function of S_{t+1}; Store A_{t+1}
  | $\tau \leftarrow t+1-n$ (τ is the time whose estimate is being updated)
  | If $\tau \geq 0$:
  | If $t+1 \geq T$:
  | $G \leftarrow R_T$
  | else
  | $G \leftarrow R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a)$
  | Loop for $k = \min(t, T-1)$ down through $\tau+1$:
  | $G \leftarrow R_k + \gamma \sum_{a \neq A_k} \pi(a|S_k) Q(S_k, a) + \gamma \pi(A_k|S_k) G$
  | $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha\left[G - Q(S_\tau, A_\tau)\right]$
  | If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is greedy wrt Q
  Until $\tau = T-1$



the 3-step tree-backup update

Desarrollo

1. Introducción

2. Predicción TD de n pasos

3. Control TD de n pasos
   3.1. Aprendizaje On-Policy de n Pasos
   3.2. Aprendizaje Off-Policy de n Pasos
      Basado en Muestreo por Importancia
      Sin Muestreo por Importancia

4. Referencias

Referencias

Silver, D. (2015). Lectures on Reinforcement Learning. https://www.davidsilver.uk/teaching/

Sutton, R., & Barto, A. (2018). *Reinforcement Learning, second edition: An Introduction*. MIT Press. https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf